

1. First java code in VSCode IDE

Create java project or standalone java extension file

Standalone java File

How to create standalone Java in VSCode: https://code.visualstudio.com/docs/java/java-tutorial#_creating-a-source-code-file

- Use this when you want to write a quick loop or test a piece of logic.
- You do not need any special configuration or folders.
- You cannot easily link it to other Java files.

Java Project

How to create Java project: <https://www.youtube.com/watch?v=tYYVeCB7Ef4>

- Use this when your program needs multiple files working together.
- It keeps your source code (src) separate from your compiled code (bin).
- It allows you to organize your files into **packages** (folders) as your code grows.

To create a Java project in VS Code, use the built-in Java project wizard.

Step-by-Step Setup

1. **Open Command Palette:** Press Ctrl + Shift + P (or Cmd + Shift + P on Mac).
2. **Launch Wizard:** Type **Java: Create Java Project** and select it from the list.
3. **Select Project Type:** Choose **No build tools** for a simple project (or select *Maven/Gradle* for advanced projects).
4. **Choose Location:** Select the folder where you want to save your project files.
5. **Name Project:** Input a name for your project folder and press **Enter**.

What Gets Created

VS Code automatically generates a standard folder structure for you.

- **src/:** The folder where you will write all your .java source code files.
- **lib/:** The folder to drop external libraries or .jar dependencies.
- **App.java:** A starter file containing a default "Hello, World!" program.

How to Run Your Project

- **One-Click Run:** Open App.java and click the **Run** button that appears directly above the main method.
- **Shortcut:** Press F5 on your keyboard to compile and debug the active file instantly

Key Differences

Feature	Standalone Java File	Java Project
Structure	A single <code>.java</code> file sitting alone.	A structured folder network (<code>src/</code> , <code>bin/</code> , <code>lib/</code>).
Use Case	Learning syntax, quick scripts, or testing code.	Building apps, software, or multi-screen programs.
External Libraries	Hard to add third-party packages or jars.	Built-in support to add external tools and packages.
Compilation	VS Code compiles only that single file.	VS Code compiles the entire application together.
Build Tools	None available.	Can use automation tools like Maven or Gradle.

Understand starting point of Java program

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Breakdown of the Code

- **public class Main:** Creates a container for your code. The class name must match your file name (e.g., Main.java).
- **public static void main:** The starting point (entry gate) where Java begins executing your program.
- **System.out.println():** The built-in command used to print text out to the screen

The **public static void main(String[] args)** line is the **exact entry point** for every Java application. When you run a Java program, the Java Virtual Machine (JVM) looks specifically for this exact sequence of words to start executing your code. If any part of it is missing or misspelled, your program will not start.

Compile the code: Type `javac Main.java` and press Enter to convert your code into machine-readable bytecode.

Execute the program: Type `java Main` and press Enter to see your message print to the console.
Or

Run from VSCode

The Java Virtual Machine (JVM)

JVM is an abstract computing machine that enables a computer to run Java programs. It acts as an intermediary runtime engine that sits between your Java code and the underlying computer hardware and operating system.

The Core Concept: "Write Once, Run Anywhere"

In traditional languages like C++, code is compiled directly into machine instructions tailored to a specific operating system (like Windows or macOS).

Java handles this differently:

1. You write Java source code (.java file).
2. The Java compiler (javac) converts your text into a universal intermediate form called **Bytecode** (.class file).
3. The **JVM** reads this bytecode and translates it on the fly into the specific machine code your current computer understands.

Because every operating system has its own specific version of the JVM, the exact same bytecode file can run on Windows, Mac, Linux, or a smartphone without any changes to the original source code.

Compiling and linking

Compiling and **linking** are the two distinct steps required to transform your human-readable source code into a functional program that a computer can actually execute. While they happen automatically when you run a build command, they perform completely different jobs.

Compiling (Translation)

Compiling is the process of translating your source code text (like a .java or .cpp file) into a lower-level format that machines or virtual machines can understand.

- **What it does:** The compiler acts as a translator. It checks your code for syntax errors, typos, and improper rules. If your code is valid, it translates it.
- **The Output:**
 - In languages like **C/C++**, the compiler generates **Object Code** (individual .obj or .o files containing binary instructions).
 - In **Java**, the compiler (javac) generates universal **Bytecode** (individual .class files).
- **Key Characteristic:** The compiler only looks at **one file at a time**. It translates the file successfully even if that file references functions or modules that live in a completely separate file.

Linking (Assembly)

Linking is the process of gathering all the individual compiled files and external libraries, gluing them together, and building one final, cohesive executable file.

- **What it does:** The linker acts as an assembler. If File A calls a function that is written inside File B, or uses a built-in system library (like Java's System.out.println), the linker goes out, finds the exact memory locations of those missing pieces, and hooks them together.
- **The Output:**

- In languages like **C/C++**, the linker outputs a single standalone executable file (like an .exe on Windows or a binary file on Mac/Linux).
- In **Java**, true linking actually happens **dynamically at runtime** by the JVM's Class Loader, though you can manually use a tool like jar to pack your compiled .class files into a single .jar package.
- **Key Characteristic:** The linker looks at the **entire project**. If you try to use a function but forgot to write it or import its library, the compiler will pass it, but the linker will crash with an "unresolved external symbol" error.

Walkthrough of the Diagram

1. **Source Code:** You write your human-readable Java logic across one or multiple files.
2. **Compiling:** The compiler translates your text files into bytecode (.class files). If you have a syntax error (like a missing semicolon), the process stops right here.
3. **Linking:** The system looks at all your separate .class files and any imported libraries. It binds them together so that when Main.class asks to use a method inside Helpers.class, the computer knows exactly where to jump in memory.
4. **Execution:** The computer safely runs your combined instructions.

